

Measuring Cost

Guide for the DPI-LS Project

Digital Performance Index — Cost Dimension (C)

Architecture, Integration & Business Capabilities

Framework: Digital Performance Index (DPI-LS)
Dimension: Cost (C) — Weight: 10%
Version: 2.5 (Extended Enterprise Edition)
Status: Architectural Reference

DPI-LS Architecture Team

Contents

1	Executive Summary & Introduction	2
2	Cost Strategy Mind Map	2
3	Mathematical Foundations of the Cost Formula	4
3.1	Dynamic Token Cost Calculation	4
3.2	Calculation Example: Token Math Breakdown	4
3.3	Deep Math: The Lemonade Stand Analogy	5
3.4	Understanding the Utilization Scale	5
3.5	Comprehensive Example: From Tokens to Final C-Score	6
4	Telemetry & Integrations	8
4.1	Table A: Cloud Infrastructure Costs	8
4.2	Table B: LLM Observability & Tokens	9
4.3	Integration Architecture: AWS Cost Explorer	9
5	Business Scenarios & Root Cause Analysis	11
5.1	Scenario 1: The Infinite Hallucination Loop (Runaway Spend)	11
5.2	Scenario 2: The Over-Provisioned Idle GPU (Poor Utilization)	12
6	Data Ingestion Payloads	13
7	Implementation Roadmap	13

1 Executive Summary & Introduction

NOTE

The **Digital Performance Index (DPI-LS)** framework quantifies AI agent performance across 7 distinct dimensions. Within this framework, the **Cost (C)** dimension tracks an agent's financial efficiency and unit economics relative to a human baseline.

Autonomous AI Agents operate at unprecedented scales and speeds. While this yields significant productivity gains, it simultaneously introduces novel financial risks. An unconstrained agent might execute thousands of hallucinated API calls, racking up severe infrastructure costs on AWS or burning through millions of tokens on OpenAI without producing a valid output.

This comprehensive technical guide outlines the architectural guidelines for measuring the Cost dimension across **both** cloud-provider billing systems (AWS Cost Explorer, GCP Billing) and deep AI observability platforms (Helicone, LangSmith, Datadog Cloud Cost Management). By combining unit economics with strict utilization penalties, DPI-LS constructs an impenetrable financial safety net for autonomous systems.

2 Cost Strategy Mind Map

The following mind map defines the core financial pillars of AI Agent Cost monitored within the DPI-LS framework.



3 Mathematical Foundations of the Cost Formula

The Cost dimension provides a normalized score indicating how financially efficient an agent operates relative to a human baseline. Cost carries a weighting of **10% (0.10)** on the final composite DPI-LS score.

IMPORTANT

Formula: $C = \min \left(1.0, \frac{\text{Human Cost Per Output}}{\text{AI Cost Per Output}} \right) \times \text{Utilization}$

- **Human Cost Per Output:** The baseline unit cost to produce one successful output (derived from human salary, benefits, and tools).
- **AI Cost Per Output:** Calculated as $\frac{\text{Total Model \& Compute Cost}}{\text{Completed Outputs}}$.
- **Utilization:** The percentage [0.0 – 1.0] of provisioned infrastructure that the agent actually utilized, penalizing over-provisioned idle instances.

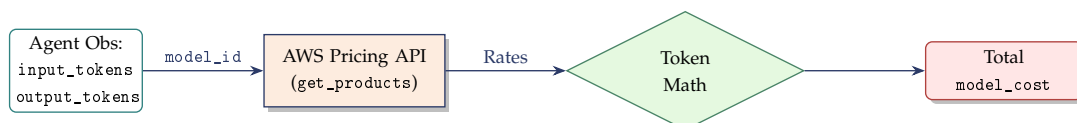
3.1 Dynamic Token Cost Calculation

Within the DPI-LS Engine, the baseline `model_cost` is not hardcoded. The `BedrockDynamicCostCalculator` class dynamically pulls foundation model pricing directly from the AWS Pricing API using `boto3`.

By mapping the `model_id` to the `input_per_1M` and `output_per_1M` USD rates, the engine accurately computes the total cost via:

IMPORTANT

Token Math: $\text{Total Cost} = \left(\frac{\text{Input Tokens}}{10^6} \times \text{Input}_{1M} \right) + \left(\frac{\text{Output Tokens}}{10^6} \times \text{Output}_{1M} \right)$



3.2 Calculation Example: Token Math Breakdown

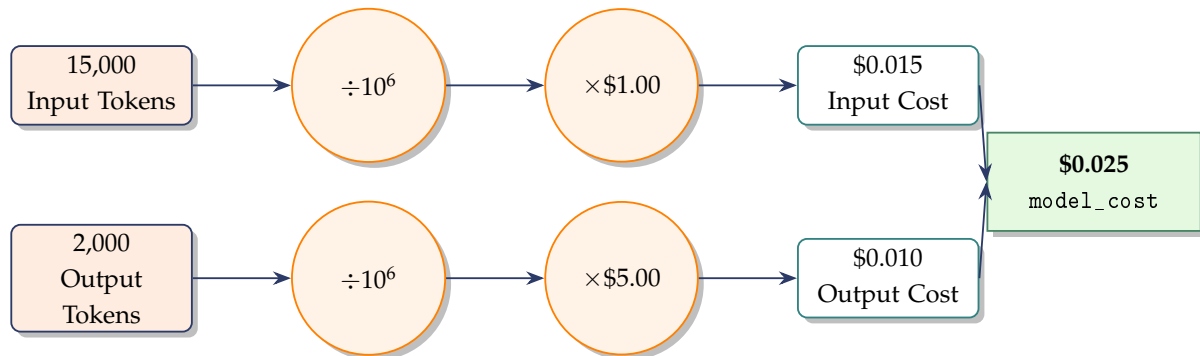
Let's walk through an exact calculation so the math is perfectly clear. Suppose an agent makes an API call using a model where the pricing is **\$1.00 per 1M input tokens** and **\$5.00 per 1M output tokens**.

The Event: The agent sends a large document (15,000 input tokens) and generates a small summary (2,000 output tokens).

$$\text{Input Cost} = \left(\frac{15,000}{1,000,000} \right) \times \$1.00 = 0.015 \times \$1.00 = \mathbf{\$0.015}$$

$$\text{Output Cost} = \left(\frac{2,000}{1,000,000} \right) \times \$5.00 = 0.002 \times \$5.00 = \$0.010$$

$$\text{Total model_cost} = \$0.015 + \$0.010 = \$0.025$$



This total of **\$0.025** is the exact number fed into the DPI-LS Engine as the `model_cost` for that specific action.

3.3 Deep Math: The Lemonade Stand Analogy

To make this math so simple a kid could understand it, let's pretend you own a lemonade stand:

- **Human Cost:** It usually costs you \$10 a day to hire your little brother to squeeze lemons. He makes 10 cups. So, your human baseline is **\$1.00 per cup**.
- **AI Cost:** You buy a giant robot to squeeze lemons. The robot costs \$20 a day, but it only makes 5 cups because it's slow. Your robot cost is **\$4.00 per cup**.
- **The Ratio ($\frac{\text{Human}}{\text{Robot}}$):** You divide the human cost (\$1) by the robot cost (\$4). This equals **0.25 (or 25%)**. This means your robot is only 25% as cost-efficient as your brother! If the robot was cheaper than your brother, the math limits the score to exactly **1.0 (100%)** so you don't get extra credit just for being cheap.

3.4 Understanding the Utilization Scale

Before calculating the final score, the ratio is multiplied by the utilization rate. This ensures that agents running on extremely expensive, underutilized infrastructure are heavily penalized.

NOTE

Where does the Utilization number actually come from?

The DPI-LS Engine does *not* magically guess your utilization. It is a manually provisioned float parameter defined inside the engine's `contract/settings.py` file (as `utilization: float = 1.0`). DevOps or FinOps teams must inject this number by pulling actual CPU/Memory usage metrics from infrastructure observability tools like **Datadog**, **Kubecost**, or **AWS CloudWatch**.

Utilization	Multiplier	Description	Infrastructure Reality
High	0.90 - 1.00	Serverless or fully saturated nodes	AWS Lambda / Optimized K8s
Medium	0.50 - 0.89	Consistent load with some idle times	Auto-scaling EC2 groups
Low	0.10 - 0.49	Severely over-provisioned infrastructure	Dedicated p4d.24xlarge GPUs
Wasteful	0.00 - 0.09	Completely idle systems bleeding money	Forgotten sandbox instances

3.5 Comprehensive Example: From Tokens to Final C-Score

Let's tie everything together. Suppose an organization defines their Human Baseline Cost as **\$15.00** per output. An AI Agent completes a task using an Auto-scaling EC2 group with **85% utilization** (0.85).

To complete one full output, the agent had to make **120 separate LLM API calls**. We will use the exact **\$0.025** token cost we calculated earlier for each API call.

Step 1: AI Cost Per Output

First, we multiply the cost of a single API interaction by the number of interactions required to finish the job:

$$\text{AI Unit Cost} = \$0.025 \times 120 \text{ calls} = \mathbf{\$3.00}$$

Step 2: The Efficiency Ratio

Next, we compare the Human Baseline against our newly calculated AI Cost:

$$\text{Cost Ratio} = \frac{\text{Human Baseline } (\$15.00)}{\text{AI Unit Cost } (\$3.00)} = \mathbf{5.0}$$

Step 3: Bounding the Ratio

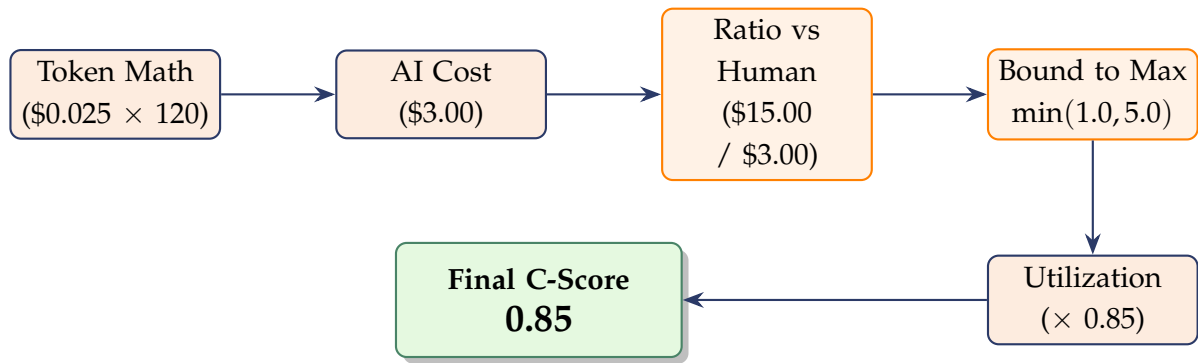
Because the agent is 5x cheaper than the human, the ratio is capped at a maximum of 1.0 (100%) so it doesn't artificially inflate the final DPI-LS score.

$$\text{Bounded Ratio} = \min(1.0, 5.0) = \mathbf{1.0}$$

Step 4: Applying Utilization for Final Score

Finally, we penalize the score by multiplying it against the actual infrastructure utilization (85%):

$$\text{Final C-Score} = 1.0 \times 0.85 = \mathbf{0.85}$$



4 Telemetry & Integrations

Cost data is rarely localized. An enterprise agent might consume tokens on OpenAI, provision vector databases on AWS, and execute lambdas. To calculate an accurate `model_cost`, the DPI-LS ingestion adapters must pull financial telemetry from diverse FinOps sources.

The tables below map how FinOps systems translate raw billing data into formalized DPI-LS Cost Data.

4.1 Table A: Cloud Infrastructure Costs

Cost Data	What It Measures	Source System	Actual Data Comes From
Unblended Resource Cost	Total API and Compute spend	AWS Cost Explorer	boto3 <code>get_cost_and_usage</code>
Idle Instance Uptime	Wasted provisioned capacity	Datadog Cloud Cost	Datadog Metrics API
Container Allocation	Kubernetes unit economics	Kubecost	K8s Allocation API
Serverless Function Duration	Cold starts & runtimes	AWS Lambda	CloudWatch Logs
Spot Instance Termination	Interruption rate & loss	AWS Spot Fleet	EventBridge
Network Data Transfer	Egress costs & bandwidth	GCP Billing	BigQuery Export
Storage Read/Write IOPS	Disk I/O cost & indexing	Azure Monitor	REST API
Database Compute Scaling	Aurora Serverless V2 ACUs	AWS RDS	CloudWatch Metrics
Storage Snapshot Costs	Backup retention & archiving	AWS S3 / Glacier	AWS Cost Explorer
Cross-Region Replication	Multi-AZ data transfer	Azure Networking	Azure Billing API
Message Queue Throughput	Kafka/SQS payload sizes	Confluent Cloud	Confluent Metrics API
Third-Party SaaS APIs	External tool consumption	Stripe / Twilio	Vendor Billing Dashboard

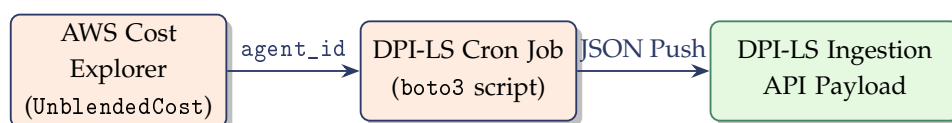
4.2 Table B: LLM Observability & Tokens

Cost Data	What It Measures	Source System	Actual Data Comes From
Input / Output Tokens	Token consumption by prompt	LangSmith / Helicone	LLM Gateway Proxy
Prompt Caching Hits	Financial savings from caches	Cloudflare AI Gateway	GraphQL Analytics API
Model Routing Spend	Cost of routing to GPT-4 vs 3.5	Finout	Finout Unit Economics
Vector DB Storage	Vector dimensions & indexing	Pinecone / Milvus	Metrics Endpoint
Text Embedding Spend	Ada V2 / Voyage tokens	OpenAI Billing	OpenAI API
Agent Iteration Cost	AutoGPT reasoning loops	AgentOps	SDK Middleware
Context Window Size	RAG document overload	Anthropic Console	Claude API
Fine-Tuning Compute	Custom model training hours	HuggingFace Endpoints	HF Inference API
Image Generation Spend	DALL-E / Midjourney API calls	OpenAI Billing	OpenAI Usage Endpoint
Audio Transcription	Whisper API minute limits	Deepgram / OpenAI	Vendor Analytics
Guardrail API Costs	Toxicity & PII detection passes	Nvidia NeMo / Lakera	Safety Gateway Proxy
Fallback Model Routing	Latency-triggered SLA retries	LiteLLM	LiteLLM Proxy Logs

4.3 Integration Architecture: AWS Cost Explorer

The reference integration relies on strict AWS Tagging policies. Every Lambda, ECS container, and Bedrock call must be tagged with `agent_id = <ID>`.

A scheduled cron job queries AWS Cost Explorer to aggregate the exact unblended cost for that specific agent.



Listing 1: cost.py - boto3 extraction by agent_id

```
1 import boto3
```

```
2 from datetime import date
3
4 ce = boto3.client("ce")
5
6 def get_specific_agent_cost(agent_id):
7     first_day = date.today().replace(day=1)
8     response = ce.get_cost_and_usage(
9         TimePeriod={
10             "Start": first_day.strftime("%Y-%m-%d"),
11             "End": date.today().strftime("%Y-%m-%d")
12         },
13         Granularity="MONTHLY",
14         Metrics=["UnblendedCost"],
15         Filter={
16             "Tags": {
17                 "Key": "agent_id",
18                 "Values": [agent_id]
19             }
20         }
21     )
22     # Extracts Total Cost
```

5 Business Scenarios & Root Cause Analysis

The following five scenarios demonstrate exactly how the Cost (C) dimension reacts to various FinOps anomalies. Each scenario provides a deep architectural breakdown, an incident timeline, and the agent's resultant math.

5.1 Scenario 1: The Infinite Hallucination Loop (Runaway Spend)

1. The Event (Financial Black Hole): An autonomous data-scraping agent is tasked with summarizing 500 web pages. Due to a parsing error on a specific Javascript-heavy site, the agent gets stuck in a recursive loop. It continuously retries the extraction, feeding the raw, messy HTML into GPT-4o over 8,000 times in a single night.

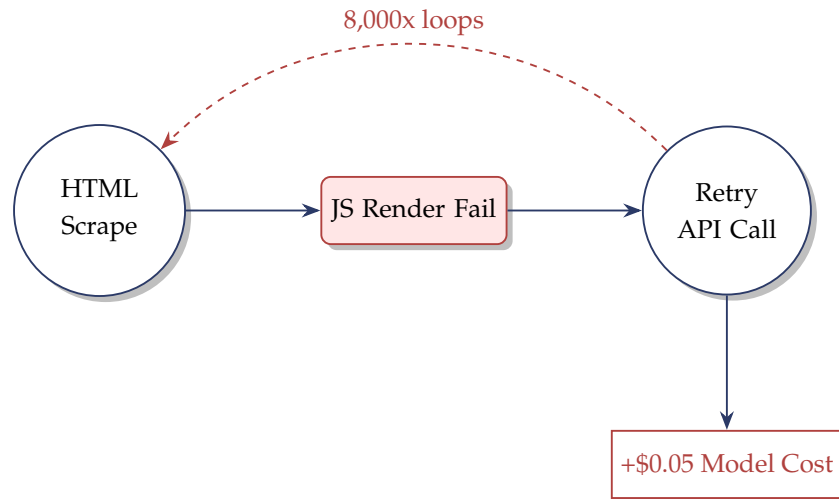
2. FinOps Impact: The `completed_outputs` remains firmly at 0, but the `model_cost` skyrockets to \$450.00. The human baseline for this task was a flat \$15.00.

3. Incident Timeline:

- **02:00 AM:** Agent encounters Javascript rendering error.
- **02:05 AM:** Agent initiates recursive retry block without a max-depth counter.
- **03:30 AM:** Helicone observability detects a massive spike in `input_tokens`.
- **06:00 AM:** AWS Cost Explorer registers \$450.00 accrued to the `agent_id` tag.
- **06:15 AM:** DPI-LS `chron` job calculates the Cost score.

4. Mathematical Resolution:

- `model_cost` = \$450.00
- `completed_outputs` = 0
- **Engine Safeguard Triggered:** Because outputs are 0, the denominator floors to 1.
- `ai_cost_per_output` = $450.0 / 1 = \$450.00$
- `human_cost_per_output` = \$15.00
- $C = \min(1.0, 15.00 / 450.00) = \mathbf{0.033}$ (3.3%)



The agent is instantly flagged as **Underperforming** and paused by the FinOps circuit breaker.

5.2 Scenario 2: The Over-Provisioned Idle GPU (Poor Utilization)

1. The Event (Wasted Capacity): An internal ML team deploys a specialized autonomous code-review agent. To ensure zero-latency response times for PR reviews, they provision a dedicated p4d.24xlarge EC2 instance. However, the engineering team only creates 2 Pull Requests over the weekend.

2. FinOps Impact: The agent successfully completes both code reviews at lightning speed, but the \$32/hour instance sat idle for 47.5 hours.

3. Mathematical Resolution:

- $\text{model_cost} = \$1,536.00$ (Weekend compute)
- $\text{completed_outputs} = 2$
- $\text{ai_cost_per_output} = \768.00
- $\text{utilization} = 0.04$ (4%)
- $C = \min(1.0, 50.00/768.00) \times 0.04 = \mathbf{0.0026}$ (0.2%)

The heavy utilization penalty completely tanks the score, signaling to the infrastructure team that Serverless / Serverless-GPU scaling is required.

6 Data Ingestion Payloads

To pipe telemetry into the DPI-LS engine, adapters must construct the canonical AgentObservation JSON. The cost block must contain three exact floats.

REST API Payload - Cost Block

```
1 {
2   "agent_id": "agent-chandra-001",
3   "timestamp": "2026-06-12T10:00:00Z",
4   "cost": {
5     "input_tokens": 1450200,
6     "output_tokens": 850000,
7     "model_cost": 45.20
8   },
9   "tasks": {
10    "completed": 1500
11  }
12 }
```

7 Implementation Roadmap

Deploying the Cost dimension across an enterprise requires strict infrastructure tagging. We recommend the following phase rollout:

1. Phase 1: Strict Tagging Policy (Weeks 1-2)

- Enforce an `agent_id` tag on every AWS, GCP, and Azure resource via Terraform/Pulumi.
- Pass the `agent_id` header in all LLM Gateway requests (e.g., Cloudflare, Helicone).

2. Phase 2: Baseline Calibration (Weeks 3-4)

- Audit current human team costs (Salary + Tools / Outputs per month).
- Inject this scalar into the DPI-LS Settings block as the `human_cost_per_output` baseline.

3. Phase 3: The FinOps Circuit Breaker (Weeks 5-6)

- Configure an automated webhook. If the DPI-LS composite score detects that the Cost (C) dimension drops below 0.40 (signaling that the AI is vastly more expensive than a human), automatically pause the agent and page the on-call engineer via PagerDuty.

